



Procédure *pas à pas* pour créer votre version personnalisée de la plateforme

Procédure pas à pas, en 7 langues, sans écrire de code.

Un guide de Pessoa Academy · Vibe Coding Workflow™

10

ÉTAPES

7

LANGUES PRÊTES

0

LIGNE ÉCRITE À LA MAIN TEMPS MOYEN

~2 h

Vous êtes le *metteur en scène*, l'IA est le dactylographe

Tout le guide repose sur une seule idée : vous décrivez ce que vous voulez en langage naturel, l'assistant IA écrit le code dans votre éditeur. Quand vous voyez une section PROMPT, copiez la phrase et collez-la dans le chat de l'assistant IA dans Visual Studio Code.

Préparez votre *boîte à outils*

POURQUOI Avant d'écrire quoi que ce soit, il vous faut une poignée d'outils, tous gratuits et désormais standard dans le monde du logiciel. Ce ne sont pas des applis au hasard : chacune a un rôle bien précis et s'emboîte avec les autres. Imaginez un atelier — **GitHub** est l'entrepôt en ligne, **Visual Studio Code** l'établi où vous ouvrez les fichiers, l'**assistant IA** votre apprenti qui écrit le code sous votre dictée, **GitHub Desktop** le chariot qui transporte les choses entre l'entrepôt et l'établi, **Supabase** le dépôt des données, **Netlify** le projecteur qui met tout en vitrine. Aujourd'hui : inscriptions et installations seulement. Pas une ligne de code.

CE QUE TU FAIS

1. Allez sur **github.com** et créez un compte gratuit (un email suffit). GitHub est le journal où chaque modification de votre code est datée et sauvegardée : si vous vous trompez, vous revenez à hier après-midi en un clic. Désormais, c'est l'ancrage auquel tout le reste va se rattacher.
2. Téléchargez **Visual Studio Code** sur code.visualstudio.com (Windows/Mac/Linux, ~100 Mo). C'est le programme avec lequel vous ouvrez, lisez et modifiez les fichiers du projet — voyez-le comme Word, mais pour du code. À la première ouverture il a l'air d'un éditeur de texte banal : la magie arrive à l'étape suivante, quand on lui greffe l'IA par-dessus.
3. Dans VS Code, cliquez sur l'icône *Extensions* dans la barre latérale gauche (quatre carrés, raccourci **Ctrl+Shift+X** sur Windows/Linux, **Cmd+Shift+X** sur Mac). Cherchez un assistant IA gratuit : le plus simple est **Gemini Code Assist** de Google, mais **Codeium**, **Continue** ou **Cody** font aussi très bien l'affaire. Installez-le et connectez-vous avec un compte Google : à partir de maintenant vous avez un chat à l'intérieur de VS Code à qui donner des consignes en français courant.
4. Téléchargez **GitHub Desktop** sur desktop.github.com et connectez-vous avec le même compte qu'au point 1. C'est une appli qui fait le pont entre votre ordinateur et GitHub : en deux clics vos modifications locales filent dans le cloud (commit + push). Sans elle, il faudrait apprendre les commandes Git en ligne de commande — on s'en passe.
5. Allez sur **supabase.com** et inscrivez-vous (avec Google ou GitHub c'est plus rapide). Supabase est l'entrepôt en ligne où votre appli stockera tout : utilisateurs, participants, mobilités, documents téléchargés. Pour l'instant il suffit d'avoir le compte ouvert — le projet réel, vous le créerez à l'étape 03, quand on aura besoin de ses identifiants.
6. Allez sur **netlify.com** et inscrivez-vous — "Sign up with GitHub" est le chemin le plus rapide, les deux services se parleront automatiquement. Netlify, c'est ce qui transforme votre code en un site web accessible à tout le monde sur internet, gratuitement. Aucun serveur à configurer, pas de FTP, rien : il recompile et republie tout seul à chaque fois que vous poussez sur GitHub.
7. Téléchargez **Node.js** en version **LTS** (Long Term Support, la version stable) sur nodejs.org. Node.js est le moteur qui fait tourner le site sur votre ordinateur pendant que vous le testez : sans lui, à l'étape 05 la commande `npm run dev` ne saurait pas quoi exécuter. L'installation est guidée — Suivant, Suivant, Terminé.

PAS D'INQUIÉTUDE POUR LES COÛTS Tous ces outils sont gratuits pour un usage personnel : aucune carte bancaire demandée pour commencer. Si à un moment on vous en demande une — arrêtez-vous et vérifiez : vous êtes sur le point d'activer une formule payante par erreur, et rien dans ce guide n'en a besoin.

02 FORK & CLONE

Faites votre *copie personnelle* du projet

POURQUOI "Forker", c'est faire une copie personnelle du projet Managemob sur GitHub : vous travaillez sur votre copie, l'original reste intact. "Cloner", c'est télécharger cette copie sur votre ordinateur pour l'ouvrir dans VS Code. Deux clics et une commande, et à la fin de cette étape vous avez le vrai projet sur votre disque, prêt à démarrer.

CE QUE TU FAIS

1. Allez sur la page GitHub du template Managemob — par ex. github.com/ETN-International/managemob-app, ou l'URL du template qui vous a été fournie. En haut à droite, cliquez **Fork** : GitHub vous demande à quel compte attribuer la copie (le vôtre) et quel nom lui donner. Choisissez quelque chose de court et en minuscules, du genre *peessoa-mobility*. En quelques secondes la copie est à vous.
2. Sur la page de votre nouveau fork, cliquez le bouton vert <> **Code** en haut à droite. Un menu s'ouvre : choisissez **Open with GitHub Desktop**. Le navigateur vous demandera l'autorisation de lancer l'appli — acceptez.
3. GitHub Desktop vous propose un dossier local pour enregistrer le projet. Acceptez le chemin par défaut ou choisissez-en un pratique (par ex. *Documents* ou *Bureau/Projets*) et cliquez **Clone**. En quelques secondes tous les fichiers du projet sont sur votre ordinateur.
4. En haut à droite dans GitHub Desktop, cliquez **Open in Visual Studio Code** : VS Code s'ouvre avec le bon dossier déjà chargé. À gauche, l'*Explorer* montre toute la structure du projet.
5. Ouvrez le chat de l'assistant IA dans VS Code et collez ce prompt : Installe les dépendances du projet. L'IA téléchargera pour vous toutes les bibliothèques dont le projet dépend (deux ou trois minutes). Quand elle vous dit qu'elle a fini sans erreur, vous êtes prêt pour l'étape suivante.

PROMPT À COLLER DANS L'ASSISTANT IA

"Explique-moi en un paragraphe ce que contient le dossier src/ — quels sont les sous-dossiers principaux et à quoi sert chacun. Ne modifie rien pour l'instant, je veux juste comprendre la carte du projet."

Préparez l'*entrepôt de données*

POURQUOI Supabase est l'entrepôt en ligne où l'appli stocke tout : utilisateurs, participants, mobilités, documents. Dans cette étape vous créez votre espace Supabase, vous récupérez les deux identifiants (URL + clé) dont le site a besoin pour lui parler, et vous créez votre premier utilisateur administrateur. Le reste — remplir la base avec les bonnes tables, verrouiller la sécurité — l'assistant IA le fera pour vous à l'étape suivante. Vous, vous cliquez.

CE QUE TU FAIS

1. Allez sur supabase.com, connectez-vous (ou inscrivez-vous si c'est la première fois) et cliquez **New project**. Choisissez un nom (par ex. *pessoa-mobility*), une **région en Europe** (Frankfurt, Ireland ou Stockholm — plus c'est proche de vos utilisateurs, plus l'appli sera réactive) et un mot de passe robuste. **Conservez ce mot de passe** dans un endroit sûr : il ne se récupère pas, et vous pourriez en avoir besoin plus tard.
2. Patientez 1–2 minutes. Supabase prépare un espace dédié rien que pour vous. C'est normal que le tableau de bord soit inactif quelques secondes — il n'est pas planté.
3. Dans la barre latérale gauche, cliquez **Project Settings** → **API**. Vous trouvez deux valeurs en clair : **Project URL** (une adresse qui commence par *https://*) et **anon public key** (une longue chaîne de caractères). Copiez les deux dans un bloc-notes provisoire : vous en aurez besoin dans un instant, à l'étape 04. Rien de technique — juste deux copier-coller.
4. Toujours dans la barre latérale, allez dans **Authentication** → **Providers**. Vérifiez que **Email** est activé (c'est généralement le cas par défaut). Pour les tests, désactivez *Confirm email* : ainsi, quand vous créerez votre utilisateur à l'action suivante, vous n'aurez pas à aller dans votre boîte mail pour cliquer un lien de confirmation. Vous le réactiverez avant le passage en production réelle (voir étape 10).
5. Sidebar → **Authentication** → **Users** → **Add user** → **Create new user**. Créez votre premier accès administrateur : un email et un mot de passe que vous retiendrez. C'est l'utilisateur avec lequel vous vous connecterez à l'étape 05 pour tester le site.

Laissez l'*assistant IA* tout brancher

POURQUOI Le site sait maintenant qu'il existe un entrepôt Supabase, mais il n'a pas encore les identifiants pour lui parler, et l'entrepôt est vide. Dans cette étape, l'assistant IA fait pour vous trois choses techniques d'un seul coup : il enregistre vos identifiants au bon endroit, remplit la base de données avec les tables de l'appli, et active la sécurité. Vous, vous collez un seul prompt et vous dites "oui" quand il vous le demande.

CE QUE TU FAIS

1. Ouvrez le chat de l'assistant IA dans Visual Studio Code (dans Gemini Code Assist c'est l'icône étoile dans la barre latérale ; dans Cursor, appuyez sur `Ctrl+L` / `Cmd+L`). Si vous voyez une option du genre *Agent mode* ou *@workspace*, activez-la : ça veut dire que l'IA peut lire et modifier les fichiers du projet.
2. Copiez ce prompt en entier (il est déjà rédigé, vous n'avez pas besoin de comprendre chaque mot) et collez-le dans le chat de l'IA :
3. Configure le projet Managemob avec ces identifiants Supabase :
URL = `https://<votre-projet>.supabase.co`
KEY = `<votre anon key>`

Trois choses, s'il te plaît : (1) crée le fichier `.env` avec ces deux variables (`VITE_SUPABASE_URL` et `VITE_SUPABASE_ANON_KEY`) et assure-toi que `.env` est dans `.gitignore` pour qu'il ne finisse jamais sur GitHub ; (2) exécute le fichier `supabase/schema.sql` sur ma base de données Supabase pour créer les tables de l'appli ; (3) active Row Level Security sur toutes les tables, pour que seuls les utilisateurs connectés puissent lire et écrire. Montre-moi étape par étape ce que tu vas faire et demande confirmation avant chaque action.

4. Remplacez `https://<votre-projet>.supabase.co` et `<votre anon key>` par les deux valeurs que vous avez copiées du tableau de bord Supabase à l'étape O3. Appuyez sur Entrée.
5. L'IA vous montre chaque opération une par une : "Je crée le fichier `.env`, ok ?", "J'exécute ce SQL, ok ?", "J'active la sécurité, ok ?". À chaque question, répondez **oui** (ou acceptez le diff). Quand elle vous dit qu'elle a fini, c'est bon : le site est connecté à Supabase, la base contient les tables, la sécurité est active — et tout cela sans que vous ayez touché un seul fichier à la main.

SI L'IA DEMANDE DE FAIRE QUELQUE CHOSE QUE VOUS NE COMPRENEZ PAS Vous pouvez tout à fait lui dire "explique-moi en français simple ce que tu vas faire, avant de le faire". C'est son rôle — vous n'avez pas besoin d'apprendre le SQL ou les variables d'environnement pour finir ce guide. Si quelque chose vous semble bizarre (par ex. elle veut supprimer des fichiers sans rapport), dites "arrête, je n'ai pas validé" et reprenez avec des consignes plus précises.

Faites tourner le site *sur votre ordinateur*

POURQUOI Avant de mettre le site en ligne, il vaut mieux le tester sur votre machine. C'est le moment où vous découvrez si Supabase est bien branché, si la connexion fonctionne, si les données se sauvegardent vraiment. Si quelque chose cloche, vous le voyez ici — sans rien casser pour personne.

CE QUE TU FAIS

1. Dans VS Code, ouvrez le chat de l'assistant IA et collez : Lance le site en local pour que je le voie dans le navigateur. L'IA exécutera la bonne commande (c'est `npm run dev`, si ça vous intéresse). Si vous préférez le faire à la main, ouvrez le Terminal (*View* → *Terminal*), tapez `npm run dev` et appuyez sur Entrée.
2. Dans le terminal, une ligne du genre Local : `http://localhost:XXXX` va apparaître. Ouvrez cette adresse dans votre navigateur (en général c'est `http://localhost:3000` ; si le port est occupé, vite en choisit un autre et vous le montre). Si l'IA l'a lancé pour vous, elle vous dit l'adresse directement dans le chat.
3. Vous verrez l'écran de connexion. Connectez-vous avec l'email et le mot de passe de l'administrateur créé à l'étape 03. S'il vous renvoie "Invalid login credentials", demandez à l'IA : La connexion ne fonctionne pas, vérifie les identifiants Supabase. Neuf fois sur dix c'est une valeur mal copiée ou un espace en trop.
4. Une fois entré, essayez de créer un participant test depuis l'accueil (juste prénom, nom, pays de destination). Sauvegardez, puis appuyez sur F5 pour recharger : si le participant est toujours là, la base de données en ligne enregistre vraiment.
5. Cliquez sur le sélecteur de langue (en haut, aussi bien sur la connexion que dans la barre latérale) : l'interface doit changer de langue instantanément dans les 7 langues. S'il manque une traduction ou si quelque chose détonne, vous le repérez ici.

SI ÇA PLANTE, PROMPT UTILE

"Si quelque chose ne fonctionne pas quand je lance le site, dis-moi ce qu'il y a dans le terminal et propose une correction simple. Ne modifie aucun fichier sans me demander avant."

Personnalisez en 7 langues

POURQUOI Managemob parle déjà 7 langues par défaut. Vous n'avez pas à traduire depuis zéro — vous devez seulement réécrire les textes visibles (libellés, titres, intitulés de boutons) pour qu'ils correspondent à VOTRE organisation, et de manière naturelle dans chaque langue. L'assistant IA s'en charge avec un seul prompt : vous choisissez le ton, l'IA l'applique aux 7 fichiers d'un coup.

CE QUE TU FAIS

1. Dans VS Code, ouvrez le chat de l'assistant IA. Vous n'avez pas à ouvrir les 7 fichiers de langue un par un — il s'en occupe.
2. Collez ce prompt (en remplaçant "Pessoa Academy" par le nom de votre organisation et en décrivant le ton souhaité) :
3. Réécris tous les textes visibles de l'application pour la marque 'Pessoa Academy', avec un ton chaleureux et professionnel (modifie ceci si tu veux un autre style). Ouvre les 7 fichiers dans src/locales/ (it.ts, en.ts, es.ts, fr.ts, de.ts, pt.ts, se.ts), modifie uniquement les valeurs (les chaînes entre guillemets), ne touche pas aux clés (les noms avant les deux-points). Traduis naturellement dans chaque langue, pas mot à mot. Montre-moi le diff avant d'appliquer.
4. L'IA vous propose toutes les modifications dans une seule revue. Parcourez le diff : si les phrases vous plaisent, validez. Si vous voulez un autre ton (plus formel, plus jeune, moins corporate), dites-le simplement — par exemple "refais en plus formel, en vouvoyant" — et elle refait le tout.

AJOUTER UNE 8E LANGUE Vous voulez une langue de plus (ex. polonais) ? Demandez à l'IA : *"Ajoute le polonais comme 8e langue de l'application, en copiant depuis en.ts et en traduisant. Ajoute aussi le drapeau dans le sélecteur de langue."* Elle le fait en 30 secondes, sans que vous touchiez au code.

Donnez-lui *votre identité visuelle*

POURQUOI L'apparence est la première chose que voient vos utilisateurs. En changeant le logo, les couleurs et le titre du site, Managemob arrête de s'appeler Managemob et devient visiblement VOTRE plateforme. Ce sont des changements purement esthétiques — la logique reste intacte — mais ils transforment complètement le ressenti.

CE QUE TU FAIS

1. Préparez votre logo en fichier `logo.svg` (préféré, vectoriel, qui s'adapte à toutes les tailles) ou `logo.png` (format carré, au moins 256×256 pixels, fond transparent). Glissez le fichier dans le dossier `public/` du projet dans VS Code : vous verrez la nouvelle icône apparaître dans l'Explorer.
2. Choisissez vos deux couleurs : la **primaire** (la couleur principale de l'appli, par ex. le bleu de votre organisation) et l'**accent** (une complémentaire pour les détails et les boutons). Si vous n'avez pas de palette toute prête, allez sur coolors.co, jouez jusqu'à ce que ça vous plaise, et notez les deux codes hexadécimaux (par ex. `#1D72B8` et `#38BDF8`).
3. Ouvrez le chat de l'IA et collez ce prompt (en remplaçant le nom de l'organisation et les codes couleur) :
4. Rebrand complet de l'application pour mon organisation 'Pessoa Academy'.
(1) Remplace le placeholder 'M' du logo par le fichier `public/logo.svg` dans `Sidebar.tsx` et `Login.tsx`. (2) Mets à jour les couleurs dans `src/styles/main.css` : `--primary = #XXXXXX`, `--accent = #YYYYYY`. (3) Change le titre du site (onglet du navigateur) en 'Pessoa Academy' dans `index.html`, et fais pointer la favicon vers mon nouveau logo. (4) Remplace le mot 'Managemob' par 'Pessoa Academy' dans les textes visibles des 7 fichiers de `src/locales/`. Montre-moi le diff avant d'appliquer chaque changement.
5. L'IA propose toutes les modifications. Validez. Rechargez le navigateur sur `localhost:3000` : votre identité visuelle est déjà appliquée.

Automatisez vos *documents*

POURQUOI L'une des choses les plus puissantes de Managemob : vous prenez n'importe quel document Word (Learning Agreement, Europass Mobility, contrat de mobilité...) et vous le transformez en un modèle qui se remplit automatiquement avec les données du participant que vous sélectionnez. La fin du copier-coller manuel : 50 participants, 50 documents, 30 secondes.

CE QUE TU FAIS

1. Ouvrez le document Word que vous voulez automatiser dans Microsoft Word, LibreOffice ou Google Docs. S'il est en PDF ou s'il s'agit d'une numérisation, il faudra d'abord le reconstruire en **.docx** — l'appli n'accepte que des fichiers **.docx**.
2. Repérez les endroits où vont les données variables (nom du participant, dates, entreprise d'accueil, pays...) et remplacez-les par des balises entre accolades. Exemples concrets :
[Prénom] [Nom] → {name} {surname}
[Date de début] → {arrival_date}
[Email] → {email}
[Entreprise d'accueil] → {host_company_name}. La liste complète des balises disponibles est sur la page *Documents* de l'appli, en cliquant "Export Placeholders".
3. Enregistrez au format **.docx** (pas **.doc**, pas **.pdf**, pas Google Docs). Depuis Word : *Enregistrer sous* → *Document Word (*.docx)*.
4. Dans l'appli Managemob : *Documents* → *Charger Modèle*. Donnez un nom au modèle (par ex. "Learning Agreement FR"), choisissez le fichier **.docx**, validez. Le modèle apparaît dans la liste.
5. Sélectionnez-le, cherchez et cochez un ou plusieurs participants dans la liste, cliquez **Générer**. Pour un seul participant vous récupérez un fichier unique ; pour plusieurs, un ZIP avec un document par personne. La mise en forme (gras, italique, tableaux) est conservée : l'appli ne remplace que les balises.

RÈGLES IMPORTANTES DES BALISES Les balises sont sensibles à la casse : écrivez {name}, pas {Name}. Pas d'espaces dans les accolades ({ name } ne marche pas). Les balises inconnues restent dans le document en texte brut ("{nomFaux}") : si votre document généré a encore des accolades, c'est que vous avez mal orthographié un nom de champ. Pour la liste exacte, allez sur *Documents* → *Export Placeholders (CSV)*.

Publiez le site *sur Netlify*

POURQUOI Pour l'instant le site ne tourne que sur votre machine et personne d'autre ne le voit. Pour le partager avec le monde, vous le mettez sur Netlify, gratuitement, branché directement à votre dépôt GitHub. À partir de ce moment, à chaque fois que vous poussez des modifications sur GitHub, Netlify reconstruit le site et le redistribue automatiquement — pas de FTP, pas de commande de déploiement, rien du tout.

A · Envoyez vos modifications sur GitHub (avec GitHub Desktop)

1. Ouvrez GitHub Desktop. Si vous avez modifié des fichiers (et après les étapes O6–O7 vous en avez modifié pas mal), en haut à gauche vous voyez la liste des fichiers changés avec un diff colorisé (vert = ajouté, rouge = retiré).
2. En bas à gauche, il y a un champ pour le message de sauvegarde : écrivez une description brève de ce que vous avez fait (par ex. "Personnalisation Pessoa Academy : logo, couleurs, textes"). Cliquez **Commit to main** : vous avez scellé un point de sauvegarde.
3. En haut, cliquez **Push origin**. Vos modifications viennent de monter sur votre dépôt GitHub — enfin sauvegardées, enfin prêtes pour Netlify.

B · Publiez sur Netlify (relié à votre dépôt)

1. Allez sur netlify.com et connectez-vous (le plus rapide est *Sign up with GitHub*). Dashboard → **Add new site** → **Import an existing project** → **Deploy with GitHub**.
2. La première fois, Netlify vous demande la permission de lire vos dépôts — accordez-la. Puis sélectionnez dans la liste le **fork** que vous avez créé à l'étape O2.
3. Netlify détecte automatiquement que c'est un projet Vite et propose les bons réglages. Vérifiez seulement : *Build command* : `npm run build` · *Publish directory* : `dist` · *Branch* : `main`.
4. Ouvrez **Show advanced** → **Add environment variable**. Ajoutez les DEUX variables que l'IA a mises dans votre `.env` à l'étape O4 : `VITE_SUPABASE_URL` et `VITE_SUPABASE_ANON_KEY` (avec les mêmes valeurs). Sans elles, l'appli en ligne ne saura pas comment parler à Supabase.
5. Cliquez **Deploy**. Le premier build prend 1–3 minutes — vous pouvez voir le log défiler en direct. Quand il affiche "Site is live", c'est terminé.
6. Netlify vous donne une adresse aléatoire du genre `https://<mots-bizarres-123>.netlify.app`. **C'est votre site en ligne !** Depuis *Site settings* → *Change site name* vous pouvez la renommer (par ex. `pessoa-mobility.netlify.app`) ; depuis *Domain management* vous pouvez brancher votre propre domaine si vous en avez un (par ex. `mobility.pessoa-academy.com`).

PROMPT À COLLER DANS L'ASSISTANT IA

"Ajoute au projet la configuration qui fait que toutes les pages internes fonctionnent correctement quand le site est en ligne sur Netlify (de sorte que, si je rafraîchis une page comme `/dashboard`, je n'aie pas l'erreur 404 Not Found). Montre-moi le fichier avant de le créer."

Vous êtes *en ligne*, dernière vérification

POURQUOI Vous êtes en ligne. Avant d'ouvrir le site aux vrais utilisateurs, on vérifie que tout fonctionne depuis l'URL publique de Netlify (et non plus localhost). Sept contrôles rapides : si tous passent, vous lancez. Si l'un échoue, il vous reste le temps de corriger — sans que personne ne s'en aperçoive.

AVANT LE LANCEMENT OFFICIEL Quand vous passez de la bêta interne aux vrais utilisateurs, faites ce ménage dans Supabase : (1) **réactivez Confirm email** dans Authentication → Providers (les nouveaux inscrits devront confirmer leur email) ; (2) **changez le mot de passe de la BD** dans Project Settings → Database (remplacez celui que vous aviez sauvegardé à l'étape 03) ; (3) supprimez l'**administrateur de test** et créez les vrais ; (4) faites une **sauvegarde** de la base (Project Settings → Database → Backups). Vingt minutes de travail qui vous sauveront la mise des dizaines de fois.

CHECKLIST FINALE

- La connexion depuis l'URL publique Netlify fonctionne (avec l'administrateur créé à l'étape 03).
- Le sélecteur de langue affiche les 7 langues et les textes changent vraiment quand je clique.
- J'ajoute un participant test, j'appuie sur F5 : s'il est toujours là, la base en ligne enregistre.
- Je génère un document Word pour un participant : les balises {name} etc. dans le fichier téléchargé sont remplacées par des données réelles.
- Le Calendar affiche le participant aux bonnes dates.
- Rafraîchir une sous-page comme /dashboard ne donne pas d'erreur 404 (= le prompt de l'étape 09 a été appliqué).
- La déconnexion me ramène sur l'écran de connexion sans erreur.

Une plateforme *à vous*, en 7 langues, sans code à la main

Visual Studio Code ouvre les fichiers. L'assistant IA écrit le code. GitHub Desktop synchronise en ligne. Supabase conserve les données. Netlify met le site en vitrine. Vous orchestrez l'ensemble et vous gardez les décisions qui comptent : nom, couleurs, textes, documents automatisés, langues. C'est le Vibe Coding Workflow™ : zéro ligne de code écrite à la main, mais le résultat est une plateforme 100 % à vous.